

PROCESS VS THREAD

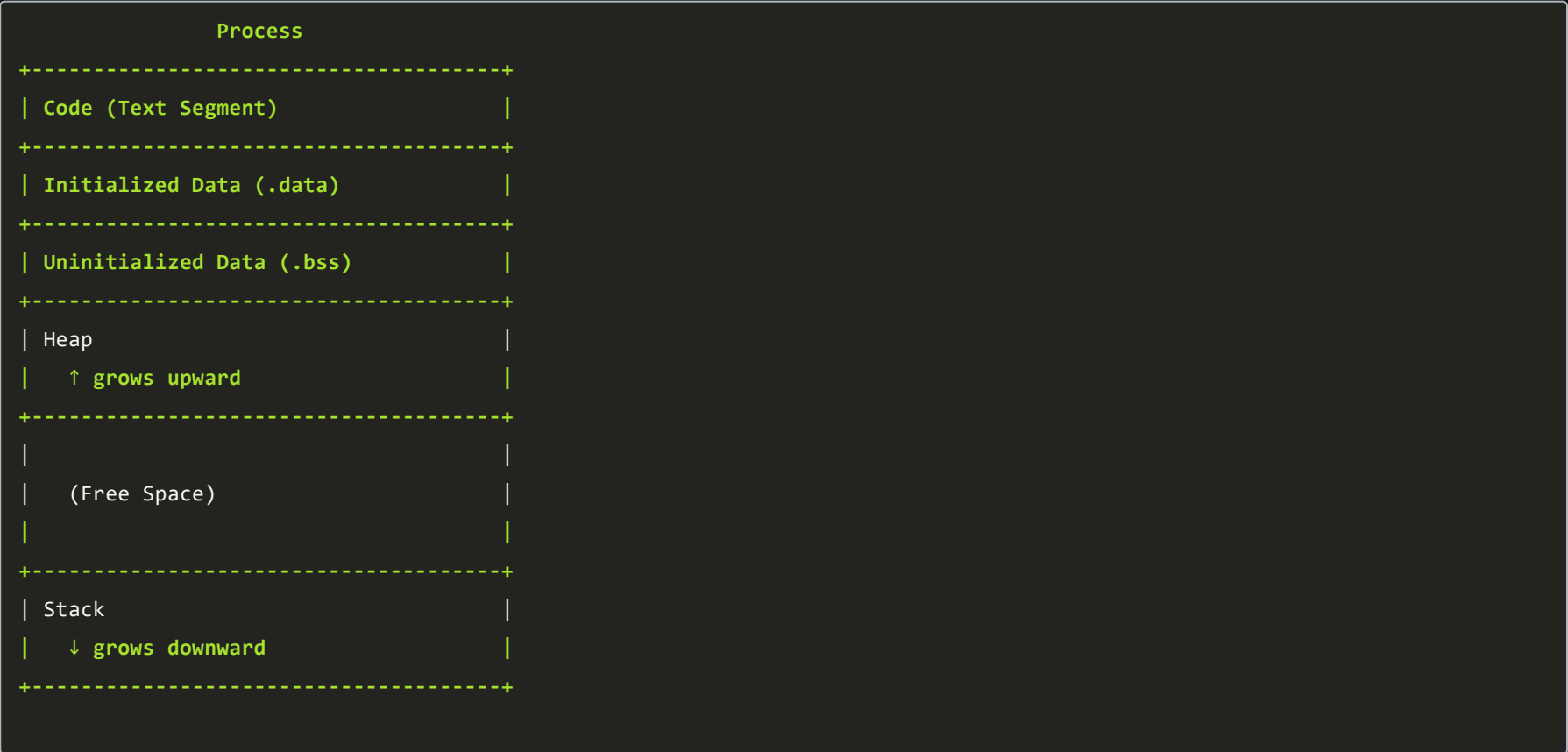
program is a file sitting on disk with some instructions on it

Interview Definition

A process is an instance of a program in execution. It is an independent execution unit with its own virtual address space, resources, and execution state. The operating system manages each process using a Process Control Block (PCB), and processes are isolated from one another for security and stability.

Once a **program starts executing**, the Operating System creates a **process**. A process consists of the program's code and all the runtime information needed to execute it.

Contents of a Process



In addition to this memory layout, the OS also maintains a **Process Control Block (PCB)**.

all the above lives in user space but PCB lives in kernel space

Imagine a Company 🏢

Process = Company

Think of a **process** as an entire company.

Each company has:

- Its own office building
- Own money
- Own files
- Own HR
- Own resources

Companies don't directly share everything with other companies.

Similarly, a **process** has its own:

- Memory
- Variables
- Heap
- Stack
- Open files
- Resources

If one company goes bankrupt, the others keep running.

Likewise, if one process crashes, the other processes are usually unaffected.

Thread = Employee

Now imagine employees inside Google.

All employees:

- Work in the same office
- Use the same meeting rooms
- Access the same company database
- Share company resources

But each employee has:

- Their own desk
- Their own notebook
- Their own task

A **thread** is exactly like an employee.

Every thread has its own:

- Stack
- Registers
- Program Counter (PC)

But all threads share:

- Process memory
- Heap
- Global variables
- Open files

Interview Answer (2–3 minutes)

A **process** is an independent program in execution . It has its own address space, memory, files, and system resources. Since processes are isolated , one process cannot directly access another process's memory, and communication between them requires Inter-Process Communication (IPC) mechanisms like pipes, sockets, or shared memory.

A **thread** is the smallest unit of execution within a process. Multiple threads of the same process share the process's code, data, heap, and open files, but each thread has its own stack, registers, and program counter . Because threads share resources, they are faster to create and switch between than processes , making them suitable for concurrent tasks. However, this shared memory also means a bug in one thread can affect the entire process.

A simple analogy is that a process is like a company, while threads are the employees working inside it. Different companies operate independently, whereas employees within the same company share common resources while performing different tasks simultaneously.